Министерство науки и высшего образования РФ
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе № 3
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Простые структуры данных»

Выполнили: ст. гр. 25ВВВ2
Денисов Р.С.
Никоноров С.Д.

Проверили: к.т.н., доцент
Деев М.В.
к.т.н., доцент
Юрова О.В.

Пенза 2026

Название

Простые структуры данных.

Цель работы

Получение практических навыков работы с базовыми и составными структурами данных (статическими и динамическими массивами, строками, структурами struct, а также связными списками), освоение механизмов выделения/освобождения динамической памяти и реализации абстрактных типов данных: приоритетной очереди, очереди (FIFO) и стека (LIFO).

Лабораторное задание

Написать программы для выполнения следующих задач:

- Задание 1: Реализация приоритетной очереди на основе односвязного списка с возможностью добавления элемента с учётом приоритета, просмотра очереди и удаления любого элемента по имени.
- Задание 2: Реализация очереди (FIFO) на основе односвязного списка с операциями добавления в конец, просмотра и извлечения из начала.
- Задание 3: Реализация стека (LIFO) на основе односвязного списка с операциями добавления на вершину, просмотра и извлечения с вершины.

Пояснительный текст и псевдокод программ

Задание 1: Приоритетная очередь

Псевдокод алгоритма:

Алгоритм Priority_Queue:

1. Инициализировать указатель head = NULL.
2. Функция push():
    - Выделить память под новый узел p.
    - Ввести имя объекта и приоритет (число > 0).
    - Если head == NULL или p->priority <= head->priority:
        - Вставить p в начало списка.
    - Иначе:

        - Пройти по списку до узла current, у которого current->next->priority >= p->priority.
        - Вставить p после current.
3. Функция review():
    - Пройти по списку от head до конца, выводя имя и приоритет каждого элемента.
4. Функция delete_any():
    - Ввести имя для удаления.
    - Найти первый узел с совпадающим именем.
    - Перестроить связи prev->next = struc->next.
    - Освободить память удалённого узла.
5. Конец алгоритма.

Пояснительный текст:

Программа реализует приоритетную очередь на основе односвязного списка. Каждый элемент хранит строковое имя (inf) и целочисленный приоритет (priority), где меньшее число означает более высокий приоритет. При добавлении элемента список поддерживается упорядоченным по возрастанию приоритета: новый узел вставляется либо в начало (если его приоритет не ниже головного), либо после первого узла с приоритетом, меньшим или равным вставляемому. Удаление любого элемента по имени выполняется с корректировкой связей между узлами и освобождением памяти. Программа устойчива к некорректному вводу: при вводе приоритета проверяется, что это число строго больше нуля.

Задание 2: Очередь (FIFO)

Псевдокод алгоритма:

Алгоритм Queue_FIFO:

1. Инициализировать указатели head = NULL, last = NULL.
2. Функция push():
    - Выделить память под новый узел p.
    - Ввести имя объекта.

- Если head == NULL: head = p, last = p.
- Иначе: last->next = p, last = p.

3. Функция review():
   - Пройти по списку от head до конца, выводя имена элементов.
4. Функция pop():
   - Если head == NULL, вывести сообщение «Список пуст» и выйти.
   - Запомнить temp = head.
   - head = head->next.
   - Если head == NULL, last = NULL.
   - Вывести имя извлечённого элемента.
   - Освободить память temp.
5. Конец алгоритма.

Пояснительный текст:

Программа реализует очередь с дисциплиной обслуживания FIFO («первым пришёл — первым вышел»). Добавление новых элементов производится в конец списка через указатель last, что обеспечивает амортизированную сложность O(1). Извлечение выполняется из начала списка через указатель head. При удалении последнего элемента указатель last сбрасывается в NULL, что предотвращает появление «висячих» ссылок. Программа корректно обрабатывает попытку извлечения из пустой очереди.

Задание 3: Стек (LIFO)

Псевдокод алгоритма:

Алгоритм Stack_LIFO:

1. Инициализировать указатель head = NULL.
2. Функция push():
   - Выделить память под новый узел p.
   - Ввести имя объекта.
   - p->next = head.
   - head = p.
3. Функция review():

- Пройти по списку от head до конца, выводя имена элементов.

4. Функция pop():
   - Если head == NULL, вывести сообщение «Список пуст» и выйти.
   - Запомнить temp = head.
   - head = head->next.
   - Вывести имя извлечённого элемента.
   - Освободить память temp.
5. Конец алгоритма.

Пояснительный текст:

Программа реализует стек с дисциплиной обслуживания LIFO («последним пришёл — первым вышел»). Все операции (добавление, извлечение) выполняются через вершину стека, роль которой играет указатель head. Добавление элемента сводится к вставке в начало списка, извлечение — к удалению первого узла. Такая реализация обеспечивает сложность O(1) для обеих операций и не требует дополнительной памяти под хранение размера стека.

Результаты работы программы

Задания 1 (Рисунок 1):

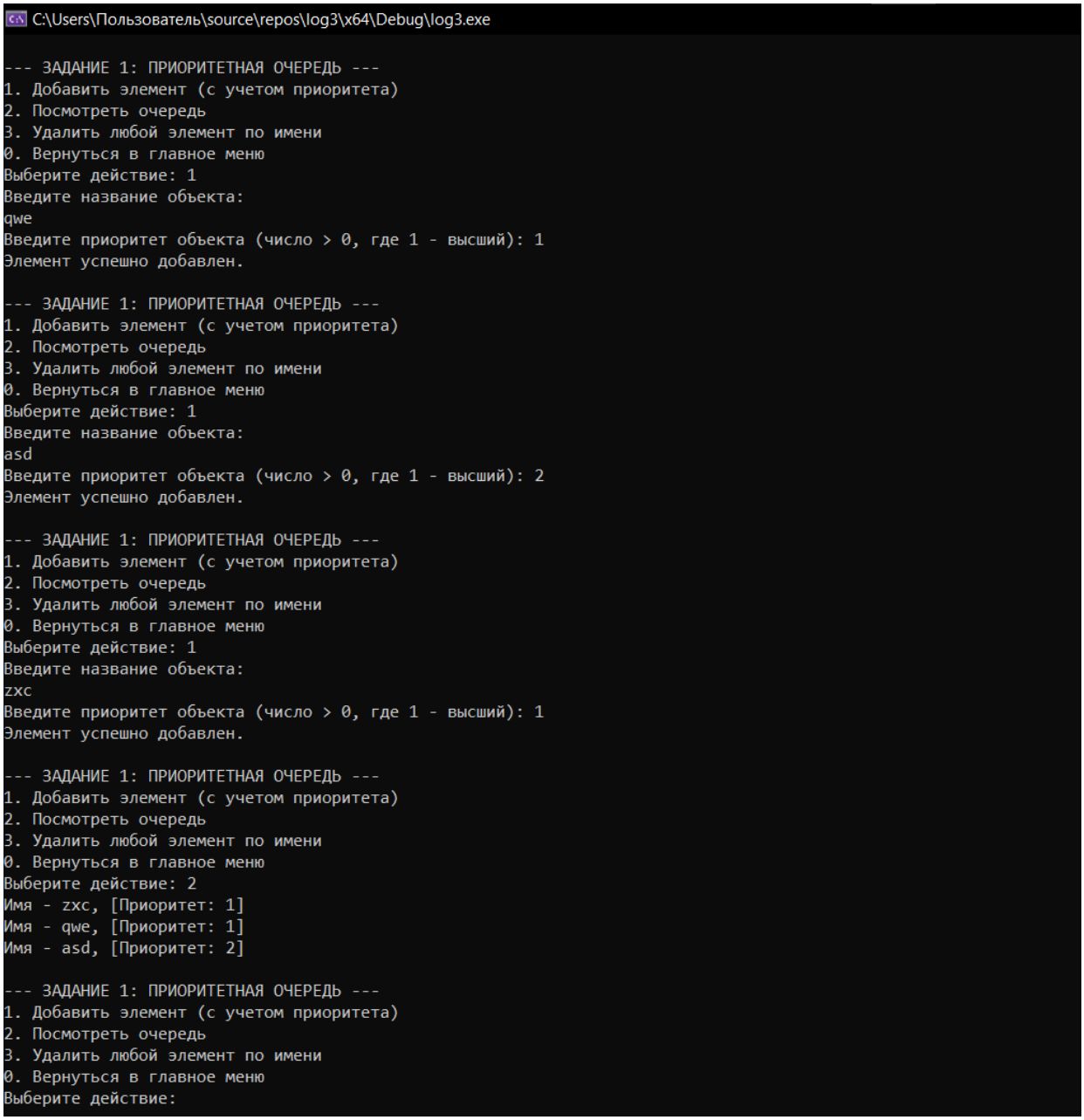


Рисунок 1.

Задание 2 (Рисунок 2):

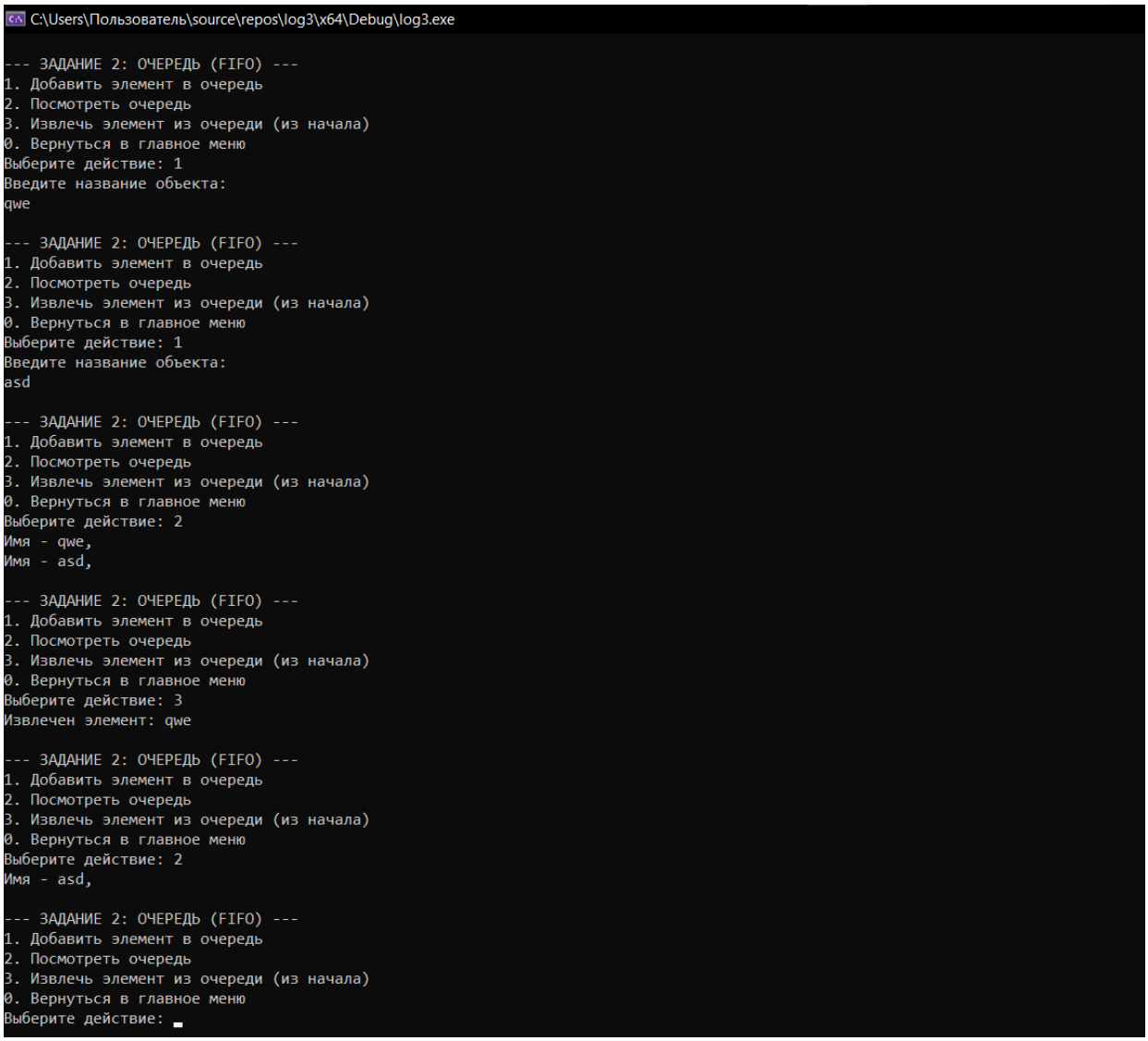


Рисунок 2.

Задание 3 (Рисунок 3):

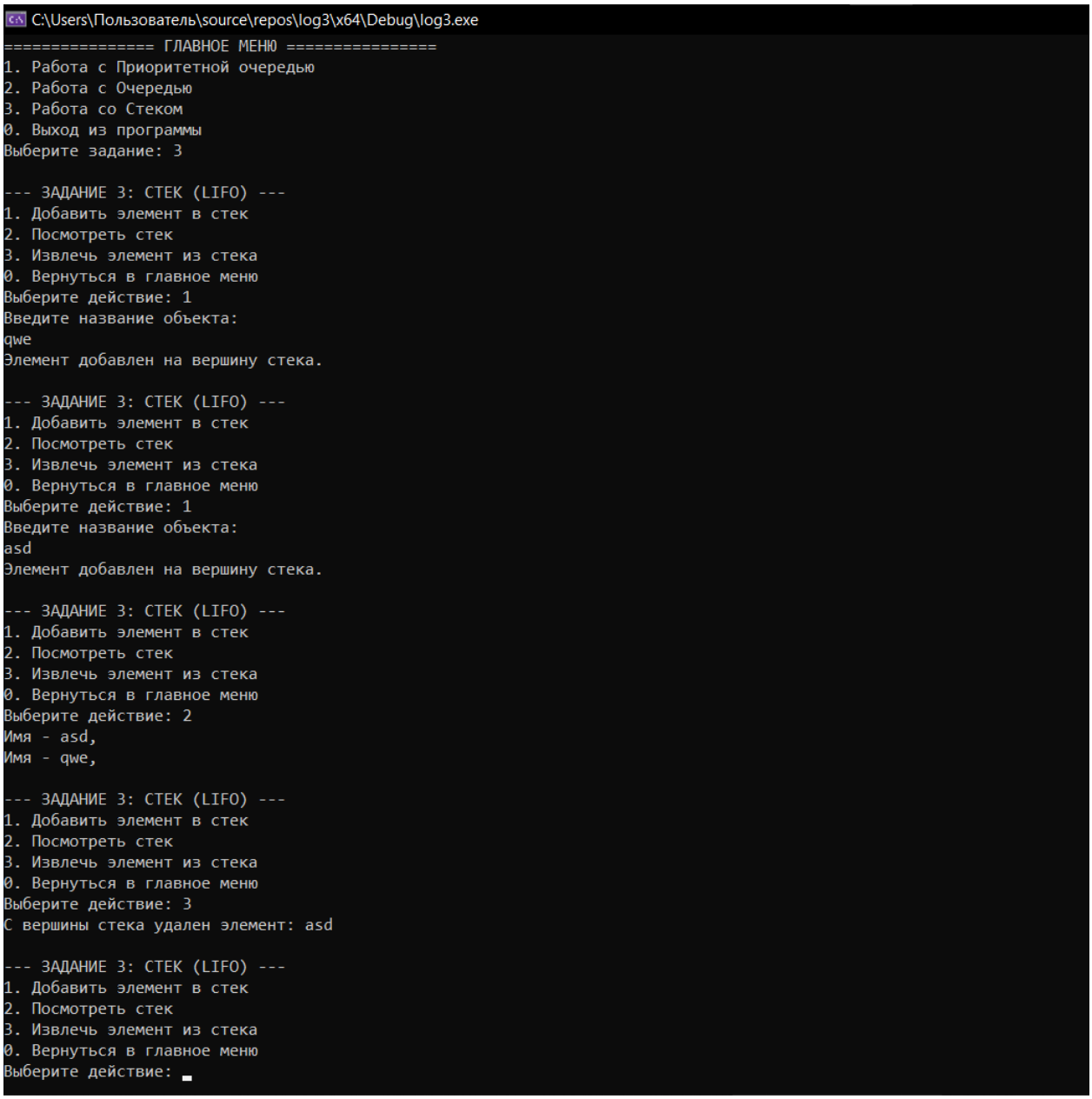


Рисунок 3

Выводы

В ходе выполнения лабораторной работы были успешно изучены и применены на практике простые и составные структуры данных языка Си. Разработаны программы, реализующие:

1. Динамическое выделение и освобождение памяти под узлы односвязных списков (malloc / free).
2. Абстрактные типы данных: приоритетную очередь, очередь (FIFO) и стек (LIFO) — на основе односвязного списка.

3. Алгоритмы упорядоченной вставки с учётом приоритета, а также операции извлечения из начала и конца списка.
4. Организацию пользовательских типов данных на базе struct, работу со строками (strcpy, strcmp) и обработку некорректного ввода.
5. Модульную структуру проекта с разделением на заголовочные файлы (*.h) и файлы реализации (*.c), а также единое меню для выбора задания.

Все разработанные программы работают корректно и без ошибок. Результаты выполнения программ полностью соответствуют теоретическим положениям и условиям практических заданий.